

fn truncate_id_bound

Michael Shoemate (@Shoeboxam)

July 26, 2026

This proof resides in “contrib” because it has not completed the vetting process.

Proves soundness of `truncate_id_bound` in `mod.rs` at commit `f5bb719` (outdated¹). `truncate_id_bound` returns an upper bound on dataset distances in terms of the symmetric distance metric.

1 Hoare Triple

Precondition

Compiler-verified

None

Caller-verified

- `id_bound.by = truncate.by`

Pseudocode

```
1 def truncate_id_bound(  
2     id_bound: Bound,  
3     truncation: Bound,  
4     total_ids: Optional[int],  
5 ) -> Bound:  
6     # Once truncated, max contributions when grouped by "over" are bounded  
7     row_bound = Bound.by(truncation.by)  
8  
9     # In each group, the worst-case row contributions is the  
10    # the number of ids contributed (known from id_bound)  
11    # times the number of rows contributed under each id (known from truncation),  
12    num_ids, num_rows = id_bound.per_group, truncation.per_group  
13    if num_ids is not None and num_rows is not None:  
14        row_bound = row_bound.with_per_group(  
15            num_ids.inf_mul(num_rows)  
16        ) #  
17  
18    # Worst case number of groups contributed is the  
19    # total number of ids contributed (total_ids)  
20    # times the number of groups contributed under each id (known from truncation).  
21    num_groups_via_truncation = None #  
22    if total_ids is not None and truncation.num_groups is not None:  
23        num_groups_via_truncation = total_ids.inf_mul(truncation.num_groups)  
24
```

¹See new changes with `git diff f5bb719..e8fa1a4 rust/src/transformations/make_stable_lazyframe/truncate/mod.rs`

```

25 # Alternatively, the number of groups contributed may be known outright from id_bound.
26 # Use the smaller of the two if both are known.
27 num_groups = option_min(num_groups_via_truncation, id_bound.num_groups)
28 if num_groups is not None:
29     row_bound = row_bound.with_num_groups(num_groups) #
30
31 return row_bound

```

Postcondition

Theorem 1.1 (Postcondition). Let g vary over groups when partitioned by `id_bound.by`. If for any two datasets x, x' we have that

$$\begin{aligned}
& \|d_{\text{SymId}}(\text{function}(x)_g, \text{function}(x')_g)\|_\infty \leq \text{id_bound.per_group}, \\
& \|d_{\text{SymId}}(\text{function}(x)_g, \text{function}(x')_g)\|_0 \leq \text{id_bound.num_groups}, \\
& d_{\text{SymId}}(\text{function}(x), \text{function}(x')) \leq \text{total_ids},
\end{aligned}$$

and `function` truncates a dataset such that,

$$\begin{aligned}
& \max_{id} \|d_{\text{Sym}}(\text{function}(x)_{id,g}, \text{function}(x')_{id,g})\|_\infty \leq \text{truncation.per_group}, \\
& \max_{id} \|d_{\text{Sym}}(\text{function}(x)_{id,g}, \text{function}(x')_{id,g})\|_0 \leq \text{truncation.num_groups},
\end{aligned}$$

then we have that

$$\begin{aligned}
& \|d_{\text{Sym}}(\text{function}(x)_g, \text{function}(x')_g)\|_\infty \leq \text{row_bound.per_group}, \\
& \|d_{\text{Sym}}(\text{function}(x)_g, \text{function}(x')_g)\|_0 \leq \text{row_bound.num_groups},
\end{aligned}$$

where `row_bound` denotes the return value.

Proof. Assume the preconditions are met, as well as the conditions of the postcondition.

We first prove the per-group bound.

$$\begin{aligned}
& \|d_{\text{Sym}}(\text{function}(x)_g, \text{function}(x')_g)\|_\infty \\
& \leq \|d_{\text{SymId}}(\text{function}(x)_g, \text{function}(x')_g)\|_\infty \cdot \max_{id} \|d_{\text{Sym}}(\text{function}(x)_{id,g}, \text{function}(x')_{id,g})\|_\infty \\
& \leq \text{id_bound.per_group} \cdot \text{truncation.per_group} \\
& = \text{row_bound.per_group} \quad \text{by line 16}
\end{aligned}$$

We now prove the number of groups bound. There are two ways to bound the number of contributed groups. We first reason by the total number of identifiers.

$$\begin{aligned}
& \|d_{\text{Sym}}(\text{function}(x)_g, \text{function}(x')_g)\|_0 \\
& \leq d_{\text{SymId}}(\text{function}(x), \text{function}(x')) \cdot \max_{id} \|d_{\text{Sym}}(\text{function}(x)_{id,g}, \text{function}(x')_{id,g})\|_0 \\
& \leq \text{total_ids} \cdot \text{truncation.num_groups} \\
& = \text{num_groups_via_truncation} \quad \text{by line 21}
\end{aligned}$$

We can also directly bound the number of contributed groups by `id_bound.num_groups`:

$$\begin{aligned}
& \|d_{\text{Sym}}(\text{function}(x)_g, \text{function}(x')_g)\|_0 \\
& = \|d_{\text{SymId}}(\text{function}(x)_g, \text{function}(x')_g)\|_0 \\
& \leq \text{id_bound.num_groups}
\end{aligned}$$

This is a valid upper bound on the number of contributed groups, because truncation is applied independently to each group.

Therefore, we take the minimum of the two upper bounds.

$$\begin{aligned}
& ||d_{\text{Sym}}(\text{function}(x)_g, \text{function}(x')_g)||_0 \\
& \min(\text{num_groups_via_truncation}, \text{id_bound.num_groups}) \\
& = \text{row_bound.num_groups} \qquad \text{by line 29}
\end{aligned}$$

In the implementation, any of the input bounds could be missing. When a bound is missing, any output bounds that require it are not claimed. $\square$